The BEAM uses schedulers to coordinate and execute functions on a CPU. By default, one scheduler is available for every logical CPU processor. So a quad-core hyper-threaded CPU will have eight schedulers. Each scheduler has a run queue that holds information about which processes are requesting execution.

Schedulers execute functions until either the process is done or a certain amount of work (called reductions) has occurred. If this happens, that process is kicked off of the scheduler and placed at the back of a run queue. This means that an infinite loop in Elixir will run forever, but it will only hold up a CPU for a fixed number of reductions at a time. This property is great for building scalable systems because a high-CPU request won't dramatically affect other requests on that server.

This explains how Elixir scales across your CPU without any work from you. By correctly using processes, you automatically get CPU scalability. And all of the major libraries use processes in the right way, so following the best practices in this book will set you up for success!

Let's take this knowledge and put it into action by implementing several different processes in Elixir.

## Explore Elixir Processes

Processes are the foundation of concurrency in Elixir. They are small, easy to spawn, and you can run as many of them as you have memory for—in production, tens to hundreds of thousands would be normal. You don't need to know much about the BEAM's process architecture to use processes, but the details highlight how powerful they are.

In addition to being scalable, processes form the foundation of durable Elixir applications. Processes are able to crash without taking down the rest of the system. This durability is one of the main traits of the BEAM.

We're going to cover the basics of processes in this section, but we'll also cover some of the interesting details of the process architecture. You'll learn how to spawn a process and pass messages to it, and you'll make an infinitely running process that responds to incoming messages. We'll also go over error isolation, memory isolation, and garbage collection.

### Spawn a Process

Elixir makes it easy to start a new process. The `spawn/1` function takes a function and executes it inside of a new process. Let's do that in IEx: